

**LAPORAN PRATIKUM STRUKTUR DATA
LINKED LIST DENGAN BAHASA JAVA**



Dosen Pengampu:
Wahyudi Dr.. S.T. M.T.

Disusun Oleh:
Kevin Maulana Sempri
2511531019

**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2025**

Kata Pengantar

Puji syukur penulis ucapkan kepada Allah SWT karena penulis dapat menyelesaikan laporan praktikum ini dengan judul "Linked List Dengan Bahasa Java" dengan tepat waktu. Laporan ini dibuat sebagai salah satu untuk memenuhi nilai praktikum pada mata Struktur Data. Laporan ini berfokus pada studi kasus Linked List dengan menggunakan Bahasa Java. Penulis menyadari bahwa laporan praktikum ini masih jauh dari sempurna, oleh karena itu penulis mengucapkan banyak terima kasih kepada Bapak / Ibu dosen dan para asisten, serta rekan-rekan mahasiswa yang telah banyak membantu penulis, dalam memberikan arahan & ilmu. Untuk perbaikan & kemajuan di masa yang akan datang, penulis mengharapkan kritik & saran yang membangun. Semoga laporan ini bisa memberikan manfaat & wawasan bagi pembaca, khususnya pada studi kasus Linked List menggunakan Bahasa Java. Akhir kata, penulis ucapkan terima kasih kepada semuanya.

DAFTAR PUSTAKA

KATA PENGANTAR	i
DAFTAR ISI	ii
DAFTAR PUSTAKA	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan Pratikum.....	1
1.3 Manfaat Pratikum.....	1
BAB II HASIL DAN PEMBAHASAN.....	2
2.1 Waktu dan Tempat.....	2
2.2 Cara Kerja.....	2
2.3 Hasil.....	3
2.4 Pembahasan	7
DAFTAR PUSTAKA.....	

Bab 1 Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi informasi menuntut adanya sistem penyimpanan data yang terstruktur dan mudah diakses. Struktur Data menjadi salah satu komponen penting dalam pengelolaan informasi, baik di bidang pendidikan, bisnis, maupun pemerintahan. Eclipse IDE merupakan salah satu perangkat lunak yang menyediakan fasilitas untuk merancang, mengelola, dan memanipulasi kode Bahasa pemrograman Java yang relatif mudah dipahami. Melalui praktikum ini, diharapkan mampu memahami Struktur Data Linked List serta mengimplementasikan studi kasus dengan menggunakan Bahasa Java.

1.2 Tujuan praktikum

1. Memahami konsep dasar struktur data dengan tipe data abstrak Linked List.
2. Mempelajari cara membuat struktur data Linked List dengan menggunakan Bahasa Java.
3. Mengimplementasikan struktur data sesuai kebutuhan studi kasus.
4. Melatih keterampilan dalam mengelola data secara sistematis dan efisien.

1.3 Manfaat praktikum

1. Memberikan pengalaman langsung dalam merancang dan mengelola struktur data.
2. Menambah wawasan tentang penggunaan struktur data dengan menggunakan Bahasa Java.
3. Mempersiapkan mahasiswa dengan keterampilan praktis yang dapat diterapkan dalam dunia kerja.
4. Membantu memahami pentingnya integritas dan konsistensi data dalam sistem informasi.

Bab 2 Hasil dan Pembahasan

Pada bab ini saya akan menampilkan hasil praktikum yang telah dilaksanakan beserta pembahasannya

2.1 Waktu dan Tempat

Praktikum ini dilaksanakan pada hari Senin tanggal 5 Mei 2026 Dilaksanakan di laboratorium Jurusan Informatika, Fakultas Teknologi Infomarsi, Universitas Andalas.

2.2 Cara Kerja

Praktikan akan membuat sebuah program yang berupa Bahasa Pemrograman Java. Perintahnya adalah membuat kode untuk Abstract Data Types (Tipe data abstrak) dan kode untuk Driver (memanggil ADT tersebut)

2.3 Hasil

Kode Java NodeSLL:

```
package pekan5_2511531019;
public class NodeSLL_2511531019 {
    //node bagian data
    int data_1019;
    //pointer ke node berikutnya
    NodeSLL_2511531019 next_1019;
    //konstruktor menginisialisasi node dengan data
    public NodeSLL_2511531019(int data_1019) {
        this.data_1019 = data_1019;
        this.next_1019 = null;
    }
}
```

Kode Java TambahSLL:

```
package pekan5_2511531019;
public class TambahSLL_2511531019 {
    public static NodeSLL_2511531019
insertAtFront_2511531019(NodeSLL_2511531019 head_1019, int value_1019) {
    NodeSLL_2511531019 new_node_1019 = new
NodeSLL_2511531019(value_1019);
    new_node_1019.next_1019 = head_1019;
    return new_node_1019;
}

    //fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511531019
insertAtEnd_2511531019(NodeSLL_2511531019 head_1019, int value_1019) {
    //buat sebuah node dengan sebuah nilai
    NodeSLL_2511531019 newNode_1019 = new
NodeSLL_2511531019(value_1019);
    //jika list kosong maka node jadi head
    if (head_1019 == null) {
        return newNode_1019;
    }
    //simpan head ke variabel sementara
    NodeSLL_2511531019 last_1019 = head_1019;
    //telusuri ke node akhir
    while (last_1019.next_1019 != null) {
        last_1019 = last_1019.next_1019;
    }
    //ubah pointer
    last_1019.next_1019 = newNode_1019;
    return head_1019;
}

    static NodeSLL_2511531019 GetNode_2511531019(int data_1019) {
        return new NodeSLL_2511531019(data_1019);
    }

    static NodeSLL_2511531019 insertPos_2511531019(NodeSLL_2511531019
headNode_1019, int position_1019, int value_1019) {
    NodeSLL_2511531019 head_1019 = headNode_1019;
    if (position_1019 < 1) {
        System.out.print("Invalid Position");
    }
    if (position_1019 == 1) {
        NodeSLL_2511531019 new_Node_1019 = new
NodeSLL_2511531019(value_1019);
        new_Node_1019.next_1019 = head_1019;
        return new_Node_1019;
    } else {
        while (position_1019-- != 0) {
            if (position_1019 == 1) {
                NodeSLL_2511531019 newNode_1019 =
GetNode_2511531019(value_1019);
                newNode_1019.next_1019 =
headNode_1019.next_1019;
                headNode_1019.next_1019 = newNode_1019;
                break;
            }
        }
    }
}
```

```

        }
        headNode_1019 = headNode_1019.next_1019;
    }
    if (position_1019 != 1) {
        System.out.print("Posisi di luar jangkauan");
    }
}
return head_1019;
}

public static void printList_2511531019(NodeSLL_2511531019 head_1019) {
    NodeSLL_2511531019 curr_1019 = head_1019;
    while (curr_1019.next_1019 != null) {
        System.out.print(curr_1019.data_1019 + " --> ");
        curr_1019 = curr_1019.next_1019;
    }
    if (curr_1019.next_1019 == null) {
        System.out.print(curr_1019.data_1019);
    }
    System.out.println();
}

public static void main(String[] args) {
    // TODO Auto-generated method stub
    //buat linked list 2 --> 3 --> 5 --> 6
    NodeSLL_2511531019 head_1019 = new NodeSLL_2511531019(2);
    head_1019.next_1019 = new NodeSLL_2511531019(3);
    head_1019.next_1019.next_1019 = new NodeSLL_2511531019(5);
    head_1019.next_1019.next_1019.next_1019 = new
NodeSLL_2511531019(6);
    //cetak list asli
    System.out.print("Senarai berantai awal: ");
    printList_2511531019(head_1019);
    //tambahkan node baru di depan
    System.out.print("tambah 1 simpul di depan: ");
    int data1_1019 = 1;
    head_1019 = insertAtFront_2511531019(head_1019, data1_1019);
    //cetak update list
    printList_2511531019(head_1019);
    //tambahkan node baru di belakang
    System.out.print("tambah 1 simpul di belakang: ");
    int data2_1019 = 7;
    head_1019 = insertAtEnd_2511531019(head_1019, data2_1019);
    //cetak update list
    printList_2511531019(head_1019);

    int data3_1019 = 4;
    int pos_1019 = 4;
    head_1019 = insertPos_2511531019(head_1019, pos_1019,
data3_1019);
    //cetak update list
    printList_2511531019(head_1019);
}
}

```

Kode Java PencarianSLL:

```
package pekan5_2511531019;
public class PencarianSLL_2511531019 {
    static boolean searchKey_2511531019(NodeSLL_2511531019 head_1019, int
key_1019) {
        NodeSLL_2511531019 curr_1019 = head_1019;
        while (curr_1019 != null) {
            if (curr_1019.data_1019 == key_1019) {
                return true;
            }
            curr_1019 = curr_1019.next_1019;
        }
        return false;
    }
    public static void traversal_2511531019(NodeSLL_2511531019 head_1019)
{
        //mulai dari head
        NodeSLL_2511531019 curr_1019 = head_1019;
        //telusuri sampai pointer null
        while (curr_1019 != null) {
            System.out.print(" " + curr_1019.data_1019);
            curr_1019 = curr_1019.next_1019;
        }
        System.out.println();
    }
    public static void main(String[] args) {
        NodeSLL_2511531019 head_1019 = new NodeSLL_2511531019(14);
        head_1019.next_1019 = new NodeSLL_2511531019(21);
        head_1019.next_1019.next_1019 = new NodeSLL_2511531019(13);
        head_1019.next_1019.next_1019.next_1019 = new
NodeSLL_2511531019(30);
        head_1019.next_1019.next_1019.next_1019.next_1019 = new
NodeSLL_2511531019(10);
        System.out.print("Penelusuran SLL: ");
        traversal_2511531019(head_1019);

        //data yang akan dicari
        int key_1019 = 30;
        System.out.print("cari data " + key_1019 + " = ");
        if (searchKey_2511531019(head_1019, key_1019)) {
            System.out.println("ketemu");
        } else {
            System.out.println("tidak ada");
        }
    }
}
```

Kode Java HapusSLL:

```
package pekan5_2511531019;
public class HapusSLL_2511531019 {
    public static NodeSLL_2511531019
deleteHead_2511531019(NodeSLL_2511531019 head_1019) {
        //jika SLL kosong
        if (head_1019 == null) {
            return null;
        }
    }
```



```

        //pindahkan head ke node berikutnya
        head_1019 = head_1019.next_1019;
        //Return head baru
        return head_1019;
    }
    //fungsi menghapus node terakhir SLL
    public static NodeSLL_2511531019
removeLastNode_2511531019(NodeSLL_2511531019 head_1019) {
    //jika list kosong, return null
    if (head_1019 == null) {
        return null;
    }
    //jika list satu node, hapus node dan return null
    if (head_1019.next_1019 == null) {
        return null;
    }
    //temukan node terakhir ke dua
    NodeSLL_2511531019 secondLast_1019 = head_1019;
    while (secondLast_1019.next_1019.next_1019 != null) {
        secondLast_1019 = secondLast_1019.next_1019;
    }
    //hapus node terakhir
    secondLast_1019.next_1019 = null;
    return head_1019;
}
    public static NodeSLL_2511531019
deleteNode_2511531019(NodeSLL_2511531019 head_1019, int position_1019) {
    NodeSLL_2511531019 temp_1019 = head_1019;
    NodeSLL_2511531019 prev_1019 = null;
    //jika linked list null
    if (temp_1019 == null) {
        return head_1019;
    }
    //kasus 1 yakni head dihapus
    if (position_1019 == 1) {
        head_1019 = temp_1019.next_1019;
        return head_1019;
    }
    //kasus 2 yakni menghapus node di tengah
    //telusuri ke node yang dihapus
    for (int i_1019 = 1; temp_1019 != null && i_1019 <
position_1019; i_1019++) {
        prev_1019 = temp_1019;
        temp_1019 = temp_1019.next_1019;
    }
    //jika ditemukan, hapus node
    if (temp_1019 != null) {
        prev_1019.next_1019 = temp_1019.next_1019;
    } else {
        System.out.println("Data tidak ada");
    }
    return head_1019;
}
    public static void printList_2511531019(NodeSLL_2511531019 head_1019)
{
    NodeSLL_2511531019 curr_1019 = head_1019;
    while (curr_1019.next_1019 != null) {

```

```

        System.out.print(curr_1019.data_1019 + "-->");
        curr_1019 = curr_1019.next_1019;
    }
    if (curr_1019.next_1019 == null) {
        System.out.print(curr_1019.data_1019);
    }
    System.out.println();
}

//kelas main
public static void main(String[] args) {
    //buat SLL 1 --> 2 --> 3 --> 4 --> 5 --> 6 --> null
    NodeSLL_2511531019 head_1019 = new NodeSLL_2511531019(1);
    head_1019.next_1019 = new NodeSLL_2511531019(2);
    head_1019.next_1019.next_1019 = new NodeSLL_2511531019(3);
    head_1019.next_1019.next_1019.next_1019 = new
NodeSLL_2511531019(4);
    head_1019.next_1019.next_1019.next_1019.next_1019 = new
NodeSLL_2511531019(5);
    head_1019.next_1019.next_1019.next_1019.next_1019.next_1019 =
new NodeSLL_2511531019(6);

    // cetak list awal
    System.out.println("list awal: ");
    printList_2511531019(head_1019);

    //hapus head
    head_1019 = deleteHead_2511531019(head_1019);
    System.out.println("List setelah head dihapus: ");
    printList_2511531019(head_1019);

    //hapus node terakhir
    head_1019 = removeLastNode_2511531019(head_1019);
    System.out.println("List setelah simpul terakhir dihapus: ");
    printList_2511531019(head_1019);

    // Deleting node at position 2
    int position_1019 = 2;
    head_1019 = deleteNode_2511531019(head_1019, position_1019);
    // Print list after deletion
    System.out.println("List setelah posisi 2 dihapus: ");
    printList_2511531019(head_1019);
}
}

```

Output Kode Java TambahSLL

```

Senarai berantai awal: 2 --> 3 --> 5 --> 6
tambah 1 simpul di depan: 1 --> 2 --> 3 --> 5 --> 6
tambah 1 simpul di belakang: 1 --> 2 --> 3 --> 5 --> 6 --> 7
1 --> 2 --> 3 --> 4 --> 5 --> 6 --> 7

```

Output Kode Java PencarianSLL

```

Penelusuran SLL: 14 21 13 30 10
cari data 30 = ketemu

```

Output Kode Java HapusSLL

```
list awal:  
1-->2-->3-->4-->5-->6  
List setelah head dihapus:  
2-->3-->4-->5-->6  
List setelah simpul terakhir dihapus:  
2-->3-->4-->5  
List setelah posisi 2 dihapus:  
2-->4-->5
```

2.4 Pembahasan

Praktikum ini menunjukkan bahwa Bahasa Java memudahkan proses membuat struktur data yang terdiri dari selector, mutator, dan lainnya. Dengan demikian, praktikum ini berhasil memberikan gambaran nyata tentang pentingnya Tipe Data Abstrak untuk struktur data dan driver sebagai eksekutor Tipe Data Abstrak untuk mendukung sistem informasi.

Bab 3 Kesimpulan

Praktikum ini berhasil menghasilkan struktur dalam data dengan menggunakan bahasa Java. Di dalam data memiliki struktur yang dapat menjalankan sebuah kasus kecil maupun besar untuk mendukung system informasi.